

# PIM Design Document

**Course:** COMP3211 Software Engineering, Fall 2026

**Project:** Command-line Personal Information Management system

**Group number:** 89

**Members and student IDs:** [Human input pending: member names and student IDs]

**Version/date:** WP05 final source 1.1, 2026-09-30

**Implementation baseline:** 4d3508db0447a3ac6346a2a2789da89208f88fd5

This document describes the architecture and public interfaces present at the frozen implementation commit. It maps the behavior in [SRS.md](#) and the product choices in [decisions.md](#) to code. The companion [technical contract](#) records the interface baseline reconciled with the frozen source, which is the authority for implementation details. Class/module names are group design choices; the official Project Description requires design explanations and diagrams but does not prescribe these names.

## 1. Introduction

The PIM manages note, task, event, and contact records in one local command-line program. The implementation isolates user interaction, in-memory behavior, and `.pim` persistence so model unit tests exercise records, CRUD, and search without terminal or file I/O. This supports the official requirement to place all model code in a separate `model` package. The frozen `src/` tree contains the working implementation and uses Python standard-library modules only.

## 2. Architecture and rationale

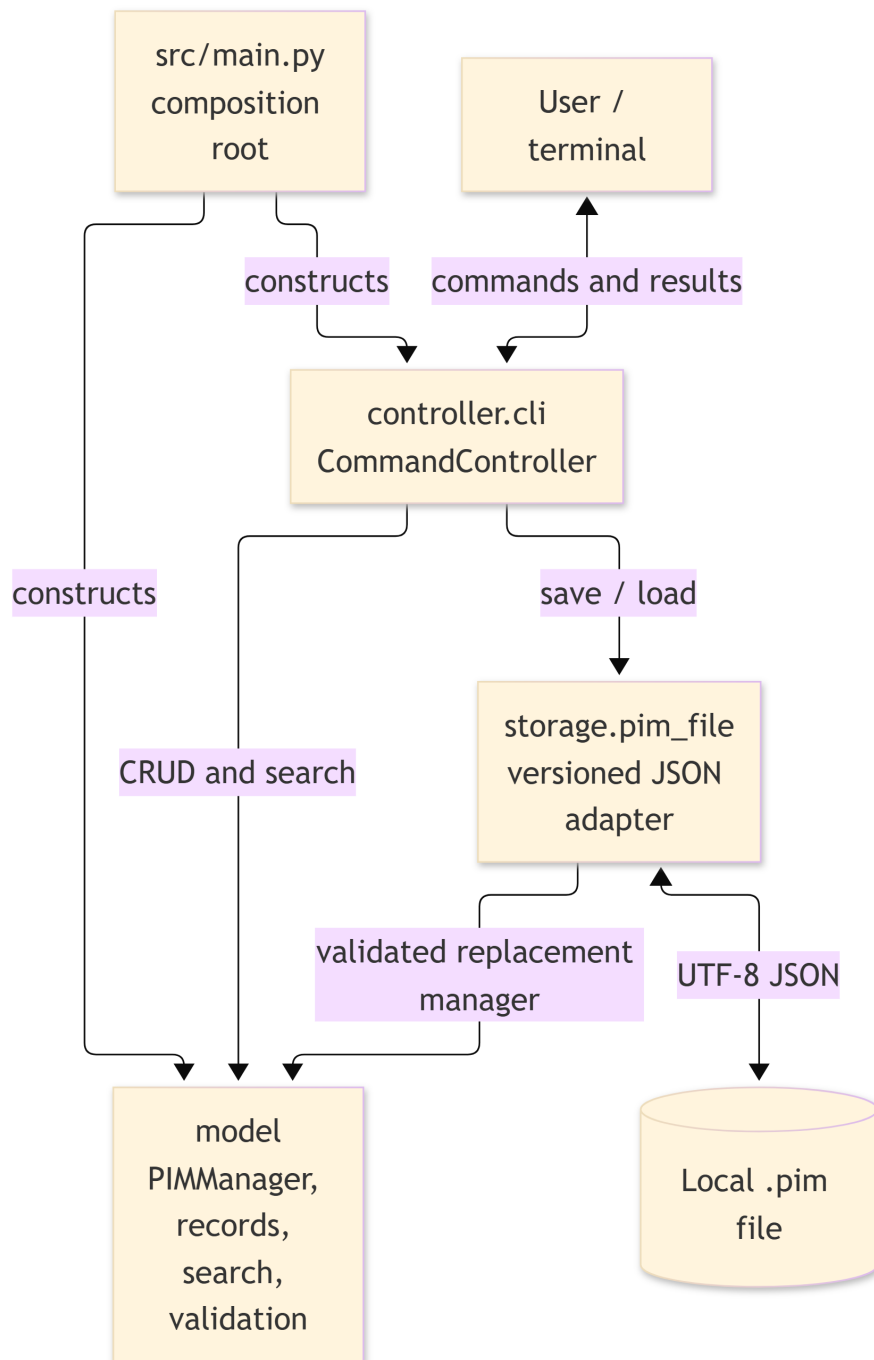


Figure 1. High-level architecture. Editable source: [architecture.mmd](#).

The generic pattern is **layered, MVC-style separation**. The terminal presentation and command interpretation form the view/controller boundary in **controller**. **model** owns PIR types, validation, ID allocation, search parsing/evaluation, and the collection. **storage** is a file adapter for versioned **.pim** snapshots. **src/main.py** only creates the initial manager and controller. This keeps the model independent of terminal and file I/O, making the official model unit-test deliverable practical. A full GUI-oriented MVC framework would add unnecessary parts to this local CLI.

Arrows in Figure 1 show allowed calls and data movement: the user talks only to the CLI; the controller invokes model operations and the file adapter; storage constructs or reads validated model state; only storage touches the local **.pim** file. On load, storage returns a **new** manager and the controller swaps its active reference only after full validation. On failure the current manager is untouched. No reused third-party architectural component is assumed; the Python standard library supplies basic dates, JSON, paths, and file replacement.

### 3. Main code components and relationships

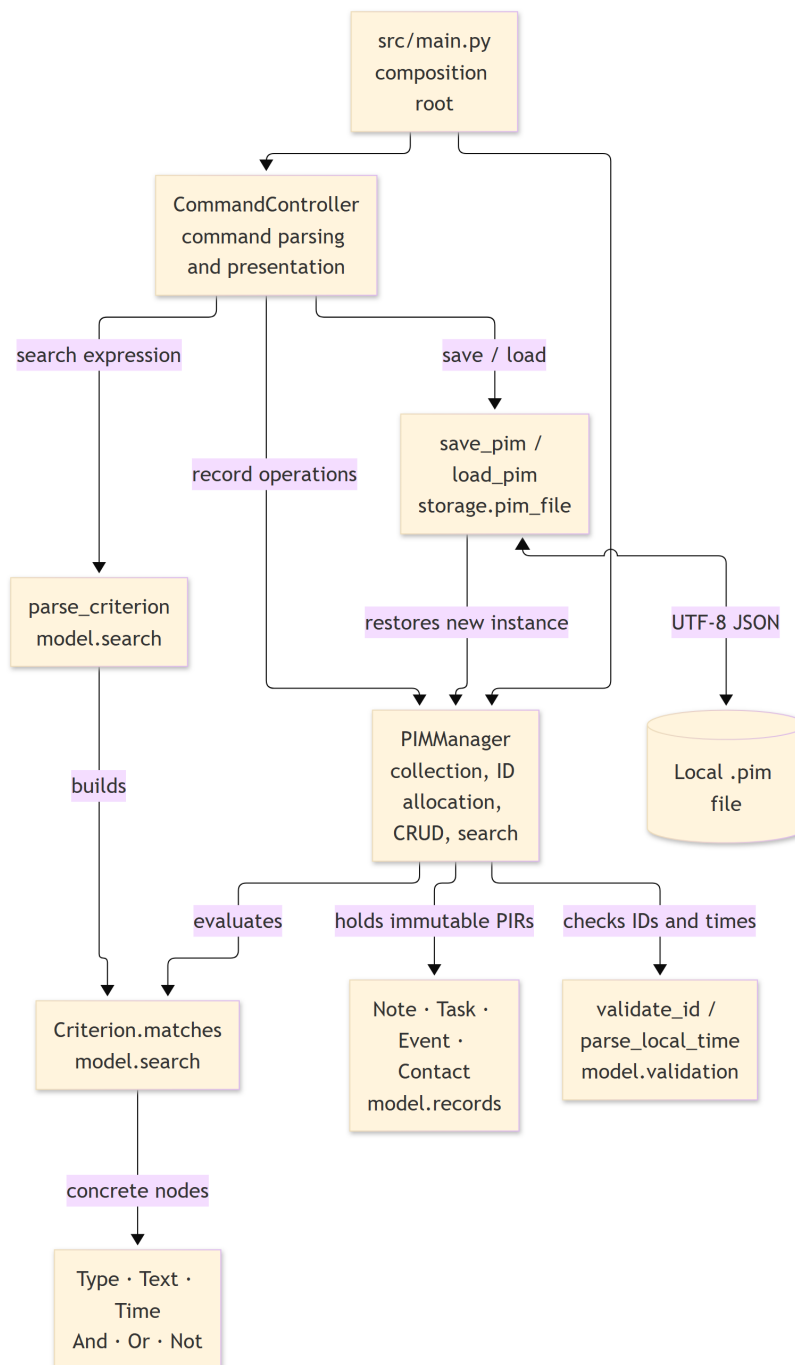


Figure 2. Main classes, modules, and their call and ownership relationships. Editable source: [components.mmd](#). Public fields and complete signatures, returns, and exceptions are detailed below.

The component diagram shows PIMManager holding four PIR classes and calling criteria through their abstract Criterion.matches interface. Dependency arrows show calls and returned state, not inheritance: the controller calls the manager, search parser, and storage; storage constructs a replacement manager. The abstract Criterion base class has immutable type, text, time, AND, OR, and NOT implementations, listed in the lower node. The parser builds those nodes without evaluating input as Python code. No reverse dependency from model to controller or storage exists in the frozen imports. The class fields, inheritance, signatures, and exceptions appear in the tables that follow so the relationships remain readable on a document page.

### 3.1 Record types and shared fields

All four records are frozen dataclasses. Each has a positive `id: int` and a read-only canonical type: `RecordType` property. Each constructor validates and trims accepted text in `__post_init__`; it rejects wrong field types, blank/multiline text, aware or non-minute datetimes, and invalid IDs with `ValidationError`. Normal creation still goes through the manager, which validates before allocating an ID. The data fields are:

| Class and constructor signature                                                                       | Data fields                                                               | Return / possible exception           |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------|
| <code>Note(id: int, text: str) -&gt; Note</code>                                                      | <code>text: str</code>                                                    | <code>Note; ValidationError</code>    |
| <code>Task(id: int, description: str, deadline: datetime) -&gt; Task</code>                           | <code>description: str, deadline: datetime</code>                         | <code>Task; ValidationError</code>    |
| <code>Event(id: int, description: str, start_time: datetime, alarm_time: datetime) -&gt; Event</code> | <code>description: str, start_time: datetime, alarm_time: datetime</code> | <code>Event; ValidationError</code>   |
| <code>Contact(id: int, name: str, address: str, mobile_number: str) -&gt; Contact</code>              | <code>name: str, address: str, mobile_number: str</code>                  | <code>Contact; ValidationError</code> |

Each dataclass also supplies generated `__init__`, equality, and representation methods. Its explicit `__post_init__(self) -> None` validates fields and may raise `ValidationError`. The read-only type properties have the signatures `Note.type -> Literal["note"]`, `Task.type -> Literal["task"]`, `Event.type -> Literal["event"]`, and `Contact.type -> Literal["contact"]`; they return the canonical type without raising. No protected methods are defined. Accepted text is trimmed at the model boundary; dates are naive local values at minute precision. `mobile_number` remains text so + and leading zeros survive. Neither past times nor the ordering of event start and alarm are restricted. Duplicate field values across records are allowed.

### 3.2 Model manager and exceptions

`PIMManager` has `_records: dict[int, Record]` and `_next_id: int`. Both are private; `next_id` is a read-only property. The manager commits a new or replacement record only after all validation succeeds, and returns ID-ascending immutable tuples for list/search. All ID arguments are positive integers; Boolean IDs are invalid. Its public signatures are:

| Method signature                                                                                                        | Return                              | Possible exception                                        |
|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------|
| <code>PIMManager() -&gt; PIMManager</code>                                                                              | Empty manager with next ID 1        | None                                                      |
| <code>next_id: int</code>                                                                                               | Current next ID                     | None                                                      |
| <code>create_note(text: str) -&gt; Note</code>                                                                          | New note                            | <code>ValidationError</code>                              |
| <code>create_task(description: str, deadline: str) -&gt; Task</code>                                                    | New task                            | <code>ValidationError</code>                              |
| <code>create_event(description: str, start_time: str, alarm_time: str) -&gt; Event</code>                               | New event                           | <code>ValidationError</code>                              |
| <code>create_contact(name: str, address: str, mobile_number: str) -&gt; Contact</code>                                  | New contact                         | <code>ValidationError</code>                              |
| <code>get(record_id: int) -&gt; Record</code>                                                                           | Selected PIR                        | <code>ValidationError, RecordNotFoundError</code>         |
| <code>list_all(record_type: str   None = None) -&gt; tuple[Record, ...]</code>                                          | All/filter matches, ID ascending    | <code>ValidationError</code>                              |
| <code>update(record_id: int, field_name: str, value: str) -&gt; Record</code>                                           | Replacement PIR                     | <code>ValidationError, RecordNotFoundError</code>         |
| <code>delete(record_id: int) -&gt; None</code>                                                                          | None                                | <code>ValidationError, RecordNotFoundError</code>         |
| <code>search(criterion: Criterion) -&gt; tuple[Record, ...]</code>                                                      | Matches, ID ascending               | <code>ValidationError</code> for a non-criterion argument |
| <code>from_rows(rows: Sequence[Mapping[str, object]], file_next_id: int, previous_next_id: int) -&gt; PIMManager</code> | Fully validated replacement manager | <code>ValidationError</code>                              |

`PIMError` is the common model exception. `ValidationError` covers invalid schema, field, time, or ID input; `RecordNotFoundError` covers an absent valid ID; `SearchSyntaxError` is a `ValidationError` for a malformed criterion. They identify the rejected input or failure category. No manager method performs terminal or file I/O. `from_rows` validates a complete snapshot without mutating the active manager, rejects duplicate/out-of-order IDs or invalid counters, and sets the new counter to `max(previous_next_id, file_next_id)`.

### 3.3 Time validation and search

| Public function or method                                                | Return                          | Possible exception              |
|--------------------------------------------------------------------------|---------------------------------|---------------------------------|
| <code>validate_id(value: object) -&gt; int</code>                        | Valid positive integer ID       | <code>ValidationError</code>    |
| <code>validate_text(field: str, value: object) -&gt; str</code>          | Trimmed, single-line text       | <code>ValidationError</code>    |
| <code>validate_datetime(field: str, value: object) -&gt; datetime</code> | Naive minute-precision datetime | <code>ValidationError</code>    |
| <code>parse_local_time(value: str) -&gt; datetime</code>                 | Naive local minute              | <code>ValidationError</code>    |
| <code>format_local_time(value: datetime) -&gt; str</code>                | Exact YYYY-MM-DD HH:mm          | <code>ValidationError</code>    |
| <code>parse_criterion(expression: str) -&gt; Criterion</code>            | Immutable criterion tree        | <code>SearchSyntaxError</code>  |
| <code>Criterion.matches(record: Record) -&gt; bool</code>                | Match result                    | None for valid criterion/record |

`Criterion` is an abstract base class, enabling `PIMManager.search` to reject non-criterion arguments with `ValidationError`. Its abstract `matches(self, record: Record) -> bool` defines the evaluation interface. The six concrete, frozen dataclass nodes are listed below. Their constructor arguments are also their public fields. Each generated constructor returns its named class and may raise `ValidationError` through `__post_init__(self) -> None`. Each class implements `matches(self, record: Record) -> bool`, with no expected exception for a parser-produced node and a valid record. A valid absent field evaluates false, and `NotCriterion` inverts that result. Type and text comparisons ignore case; time comparisons use actual date/minute order.

| Concrete constructor / public fields                                                                                      | Evaluation behavior                                     |
|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <code>TypeCriterion(value: RecordType) -&gt; TypeCriterion</code>                                                         | Compare the canonical PIR type.                         |
| <code>TextCriterion(field: TextField, needle: str) -&gt; TextCriterion</code>                                             | Case-insensitive substring on an applicable text field. |
| <code>TimeCriterion(field: TimeField, operator: Literal["&lt;", "&gt;", "="], value: datetime) -&gt; TimeCriterion</code> | Chronological comparison on an applicable time field.   |
| <code>AndCriterion(left: Criterion, right: Criterion) -&gt; AndCriterion</code>                                           | Match when both children match.                         |
| <code>OrCriterion(left: Criterion, right: Criterion) -&gt; OrCriterion</code>                                             | Match when either child matches.                        |
| <code>NotCriterion(operand: Criterion) -&gt; NotCriterion</code>                                                          | Negate its child's result.                              |

These node constructors are also usable directly in model tests. Their `__post_init__` methods canonicalize valid type/field names and raise `ValidationError` for an invalid type/field, empty text needle, invalid time operator/value, or non-`Criterion` child. A text needle is not trimmed. Parser-specific quote validation occurs before construction; the parser translates node validation faults to `SearchSyntaxError`. This keeps parser-produced and directly constructed criteria subject to the same semantic rules.

The tokenizer recognizes quoted literals and only `\` and `\\` escapes, field/type identifiers, comparison and Boolean operators, and parentheses. The parser checks the entire expression and creates the criterion tree before `PIMManager.search` evaluates any PIR. The frozen `_Parser.parse` uses explicit operator and criterion-value stacks to apply parentheses, `!`, `&&`, then `||` precedence; repeated binary operators group left to right. `_evaluate` uses an explicit work stack and short-circuit rules for Boolean nodes. Neither routine calls `eval()` or relies on the Python call stack to traverse nested expressions. `parse_local_time` uses parsing plus round-trip formatting to enforce the exact zero-padded external syntax. The model performs this conversion even if the controller passes a valid-looking string, so model tests do not depend on the CLI.

### 3.4 Storage, controller, and entry point

| Public function or method                                                                              | Return                                                                  | Possible exception                                                                                           |
|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>PersistenceError(category: str, detail: str) -&gt; PersistenceError</code>                       | Exception carrying category and message text                            | None for string arguments                                                                                    |
| <code>save_pim(manager: PIMManager, path: Path) -&gt; None</code>                                      | None                                                                    | <code>PersistenceError</code>                                                                                |
| <code>load_pim(path: Path, current_next_id: int) -&gt; PIMManager</code>                               | New fully validated manager                                             | <code>PersistenceError</code>                                                                                |
| <code>CommandController.__init__(self, manager: PIMManager, working_directory: Path) -&gt; None</code> | Initialized controller holding the active manager and working directory | <code>ValidationError</code> for invalid constructor inputs                                                  |
| <code>CommandController.run(input_stream: TextIO, output_stream: TextIO) -&gt; None</code>             | None at exit or EOF                                                     | I/O exceptions from the supplied streams may propagate; handled command errors are printed and do not escape |
| <code>main() -&gt; None</code>                                                                         | None                                                                    | Unexpected setup/stream errors may propagate                                                                 |

Storage owns UTF-8 JSON decoding/encoding, exact format/version/key checks, record serialization, and same-directory temporary-file replacement. It rejects malformed files and returns a new manager only after full validation. A failed save leaves an existing destination intact; a failed load leaves the current manager and its counter intact. `PersistenceError` names a safe category such as path, format, or I/O. The version 1 JSON schema and failure rules are fully specified in the technical contract.

The controller owns `_manager: PIMManager` and `_working_directory: Path`. It accepts the SRS command vocabulary, shows `pim>`, resolves relative paths, and turns expected model, storage, and `CommandSyntaxError` failures into `Error: CATEGORY: DETAIL` followed by another prompt. The fixed categories are `COMMAND`, `VALIDATION`, `NOT_FOUND`, `SEARCH`, `PATH`, `FORMAT`, and `IO`, mapped by exception type or storage failure category as specified in the technical contract. It formats complete record details and ID-ordered list/search summaries. It parses an ID token as positive decimal digits, handles `exit` and `EOF`, and never performs model validation in place of the model. `main()` only composes these objects and passes standard input/output. The explicit stream parameters permit CLI tests without patching global streams.

### 3.5 Internal method relationships

Python's leading underscore marks the following implementation methods as non-public. They are included because they explain how the main classes collaborate; callers should use the public entry points above. The stated exceptions are those produced for valid internal call shapes with potentially invalid user data.

| Method and argument types                                                                                                    | Return                                       | Possible exception / role                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PIMManager._create(record_type: str, *values: object) -&gt; Record</code>                                              | New PIR                                      | <code>ValidationError</code> from time or record validation; commits only after a valid record exists.                                                  |
| <code>CommandController._path(value: str) -&gt; Path</code>                                                                  | Absolute or working-directory-relative path  | None for a string argument; storage validates the <code>.pim</code> suffix.                                                                             |
| <code>CommandController._print_records(records, output_stream: TextIO, empty: str, detailed: bool = False) -&gt; None</code> | None                                         | Supplied stream I/O errors may propagate; renders summaries or details.                                                                                 |
| <code>CommandController._dispatch(line: str, output_stream: TextIO) -&gt; bool</code>                                        | True only for exit                           | <code>CommandSyntaxError</code> , model errors, <code>PersistenceError</code> , or supplied stream I/O errors; run catches the expected command errors. |
| <code>_Parser.__init__(tokens: list[tuple[str, str]]) -&gt; None</code>                                                      | Parser state with tokens and position fields | None for token lists from <code>_tokenize</code> .                                                                                                      |
| <code>_Parser._peek() -&gt; tuple[str, str] \   None</code>                                                                  | Current token, if present                    | None.                                                                                                                                                   |
| <code>_Parser._take(kind: str \   None = None, value: str \   None = None) -&gt; str</code>                                  | Consumed token text                          | <code>SearchSyntaxError</code> when the expected token is absent.                                                                                       |

| Method and argument types                           | Return                              | Possible exception / role                                                                                                         |
|-----------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>_Parser._accept(value: str) -&gt; bool</code> | Whether matching token was consumed | None.                                                                                                                             |
| <code>_Parser.parse() -&gt; Criterion</code>        | Validated criterion tree            | <code>SearchSyntaxError</code> ; <code>parse_criterion</code> also translates node <code>ValidationError</code> to this category. |
| <code>_Parser._atom() -&gt; Criterion</code>        | Type, text, or time criterion       | <code>SearchSyntaxError</code> or a node <code>ValidationError</code> , translated by <code>parse_criterion</code> .              |

The module helpers `_tokenize(expression: str) -> list[tuple[str, str]]` and `_evaluate(root: Criterion, record: Record) -> bool` respectively produce `SearchSyntaxError` for bad lexical input and a Boolean for a valid node and record. Storage's underscored helpers validate paths and JSON object keys; they are called only by `save_pim` or `load_pim`. No external caller depends on those helpers.

## 4. Search, select, and update example

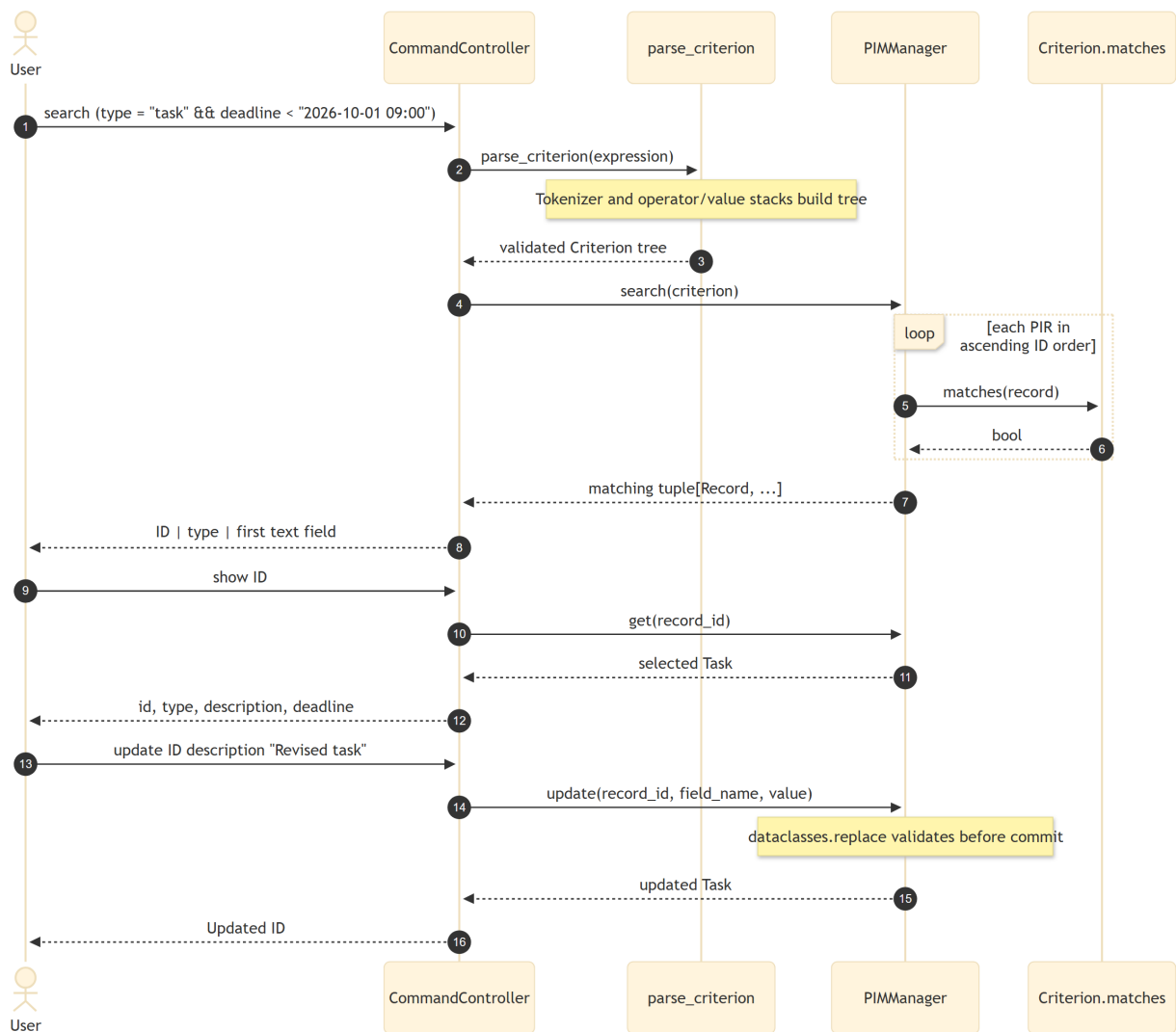


Figure 3. Example collaboration. Editable source: [search-update-sequence.mmd](#).

The user enters `search (type = "task" && deadline < "2026-10-01 09:00")`. `CommandController._dispatch` extracts the expression; `parse_criterion` tokenizes it and builds a validated criterion tree with explicit stacks; `PIMManager.search` evaluates it over the ID-ordered collection through `Criterion.matches`. The controller shows ascending-ID summaries. The user chooses an ID and requests full details with `show ID`, then changes one field with `update ID description "Revised task"`. `PIMManager.update` gets the selected record, validates a dataclass replacement, and commits it only after validation succeeds. The diagram includes the course-requested criterion definition, search, selection, and update path. It shows the successful

path; invalid expressions stop before search, and failed updates leave the selected PIR unchanged.

## 5. Requirement allocation and design checks

| SRS IDs       | Design owner                                                                                        |
|---------------|-----------------------------------------------------------------------------------------------------|
| FR-01–FR-11   | Record classes, validation, manager creation and ID counter                                         |
| FR-12–FR-15   | Controller command tokenizer and dispatch                                                           |
| FR-16–FR-17   | Manager one-field update                                                                            |
| FR-18–FR-25   | Manager list/get/delete; controller rendering, help, session lifecycle                              |
| FR-26–FR-40   | Search tokenizer/parser, criterion nodes, manager evaluation, controller summaries                  |
| FR-41–FR-48   | Storage and manager restoration; controller reference swap and path resolution                      |
| FR-49–FR-50   | Controller error display and validation-before-commit in every mutating boundary                    |
| NFR-01–NFR-05 | Local CLI, Python version, standard-library imports, separate model and identifiable other packages |

The ranges are inclusive and allocate every FR-01–FR-50 and NFR-01–NFR-05. At the frozen commit, tests in `tests/model/`, `tests/storage/`, `tests/controller/`, and `tests/test_acceptance.py` exercise the implemented paths. The separate WP05 coverage reports trace each requirement and quote the WP04 test results. Python 3.11, included in NFR-02, was unavailable for verification and remains an evidence gap.